Simulacro RA1–RA2 · Ficheros y JDBC (Variante)

Prueba escrita de entrenamiento, variante del simulacro T1 y T3 (Ficheros y JDBC). Mismo formato: tres ejercicios prácticos más cuestiones teóricas integradas. No se usa el ordenador durante la resolución; al terminar se puede comprobar el código. Se valoran la coherencia del programa, la gestión de excepciones y la documentación.

1. Persistencia JSON con Jackson. Catálogo de películas (4,5 puntos)

Desarrolla una aplicación sencilla que permita guardar y recuperar una lista de películas exclusivamente en formato JSON (sin CSV).

Define `Pelicula` con `titulo`, `director`, `anio` y `duracionMinutos`; constructor vacío, constructor con argumentos, getters, setters y `toString()`.

Desde `Principal`, crea un `ArrayList<Pelicula>` con cuatro películas y escribe la lista completa en `catalogo.json` mediante `ObjectMapper.writeValue()`.

Lee el mismo fichero mediante `ObjectMapper.readValue()` y `TypeReference<List<Pelicula>>`, recorre la lista recuperada y muestra los datos por consola, ordenados por año de forma ascendente.

Gestiona las excepciones correspondientes (`IOException` y derivadas), verifica que se recuperan las cuatro películas y documenta el código brevemente.

Variante CSV (opcional, para practicar 1.e): repite la lectura/escritura usando un fichero `catalogo.csv` (una línea por película, campos separados por `;`) con `BufferedReader` / `BufferedWriter`, y compara en un comentario qué ventajas tiene el JSON frente al CSV para este caso (tipos de datos, listas anidadas, legibilidad).

2. Fichero binario de objetos. Vehículos (3 puntos)

Desarrolla una aplicación Java sencilla que guarde y recupere objetos `Vehiculo` en `vehiculos.dat` mediante serialización.

Define `Vehiculo` con `matricula`, `marca` y `kilometraje`, constructor con parámetros, getters y `toString()`. Implementa `Serializable`.

En `Principal`, crea tres objetos `Vehiculo` con datos introducidos por teclado y guárdalos con `ObjectOutputStream` y `writeObject()`.

Abre `vehiculos.dat` con `ObjectInputStream`, recupera los tres objetos con `readObject()` y muestra sus datos. No se exige menú ni modificar registros.

Gestiona `IOException` y `ClassNotFoundException`, cierra los flujos y comenta brevemente el código; comprueba que los datos leídos coinciden con los guardados.

Variante texto plano (opcional, para comparar acceso secuencial): añade un método que, en vez de serializar, escriba los mismos datos como texto en `vehiculos.txt` (una línea por vehículo, campos separados por coma) usando `BufferedWriter`, y otro que los lea con `BufferedReader` y los reconstruya como objetos `Vehiculo`. Explica en un comentario qué ventaja tiene cada formato (binario vs. texto) para este caso.

3. JDBC con MySQL y clase DAO. Listado de empleados (2,5 puntos)

Utiliza la tabla `EMPLEADOS` ya creada en un contenedor Docker con MySQL: `EMPLEADOS(id INT, nombre VARCHAR(50), departamento VARCHAR(50), salario DECIMAL(10,2))`. La tabla tiene registros y el driver JDBC está disponible en el proyecto.

Desarrolla dos clases: `EmpleadoDAO`, que encapsula el acceso JDBC a MySQL (`jdbc:mysql://localhost:3306/empresa`, usuario `app_user` y contraseña obtenida de `DB_PASSWORD`), y `Principal`, que gestiona el teclado y muestra la información.

En `EmpleadoDAO` implementa `listarPorSalarioMinimo(double salarioMinimo)`: abre la conexión, prepara `SELECT id, nombre, departamento, salario FROM EMPLEADOS WHERE salario >= ?` mediante `PreparedStatement`, asigna el parámetro y recorre todas las filas del `ResultSet`. Devuelve un `String` con los empleados encontrados o un mensaje si no hay resultados.

Desde `Principal` pide al usuario un salario mínimo, llama al método del DAO y muestra su resultado. Cierra `Connection`, `PreparedStatement` y `ResultSet`; gestiona `SQLException` y añade comentarios breves que expliquen la separación de responsabilidades.

No se exige una clase de entidad `Empleado`, ni `ArrayList`, ni menú, ni interfaz gráfica. No debes crear tablas ni insertar, modificar o borrar registros.

Variante (opcional, para practicar transacciones): añade a `EmpleadoDAO` un método `aumentarSalario(int id, double incremento)` que haga un `UPDATE` dentro de una transacción explícita (`setAutoCommit(false)`, `commit()` / `rollback()` en el `catch`), para practicar el bloque E de la teoría.